

Can LLMs Extract Architectural Design Decisions from Source Code Commits? – A Preliminary Exploratory Study

Amey Karan¹, Rudra Dhar¹, Mohamed Soliman², and Karthik Vaidhyanathan¹

¹ International Institute of Information Technology Hyderabad, India
 {amey.karan, rudra.dhar}@research.iiit.ac.in,
 karthik.vaidhyanathan@iiit.ac.in

² Paderborn University, Germany mohamed.soliman@uni-paderborn.de

Abstract. Context: Architectural Design Decisions (ADDs) capture the rationale behind the structure and evolution of software systems but are rarely documented explicitly, and are often hidden inside source code commits. Recovering them is important for Architectural Knowledge Management (AKM).

Problem: Extracting ADDs from commits is challenging due to their implicit and unstructured nature. Large Language Models (LLMs) have shown strong capabilities in understanding code and text, yet their effectiveness for this task remains underexplored.

Study: We present a preliminary study using four LLMs (Gemini 3 Pro, DeepSeek R1, Kimi K2, Qwen3) with zeroshot and fewshot prompting on 30 developer-written ADDs from open-source projects. We score outputs with ROUGE-L, BLEU, METEOR, and BERTScore, and one author manually reviews the Gemini outputs.

Results: All models reach a BERT-F1 above 0.81, and fewshot prompting improves alignment (Gemini BERT-F1: 0.828 $\rightarrow$ 0.847). However, the generated ADDs are often too long, implementation-focused, and miss the rationale behind the decision. This highlights opportunities for architecture-aware LLM systems and automated AKM.

Keywords: Architectural Design Decisions · Code commits · Large Language Models

1 Introduction

Software architecture can be seen as a set of Architectural Design Decisions (ADDs) that shape a system’s structure, behaviour, and evolution [11]. ADDs are at the heart of Architectural Knowledge Management (AKM), but are rarely written down [3,21]. Instead, architectural information is spread across artifacts like issue trackers, mailing lists, blogs, and source code commits [18,20,2,17].

Existing extraction approaches mostly rely on rich textual artifacts such as issue trackers and mailing lists, which are often incomplete or missing, leaving much architectural knowledge unrecovered [21,2]. Source code commits are the

most consistently available artifact of how a system evolves and are known to carry architectural information [18,2]. However, commits primarily describe low-level implementation modifications, whereas ADDs are typically expressed at a higher level of abstraction. Therefore, recovering ADDs from commits requires identifying architecturally significant changes and interpreting their broader design intent and rationale.

Large Language Models (LLMs) work well on code and natural language [8,4,12]. This makes LLMs a promising candidate for extracting architectural knowledge from software repositories. However, it is not clear how well they can recover ADDs from commits, particularly when architectural rationale and context are only implicitly represented.

In this paper, we present a preliminary exploratory study on extracting ADDs from commits using LLMs. We compare zeroshot and fewshot prompting on four LLMs using 30 developer-written ADDs from open-source projects, yielding 60 generated decisions. These are scored using ROUGE-L, BLEU, METEOR, and BERTScore. To complement the metrics, an author also manually reviewed the outputs of the best model (Gemini 3 Pro) along three dimensions – *abstraction level*, *verbosity*, and *rationale*. All models reach a BERT-F1 above 0.81 and few-shot prompting improves alignment, but outputs are often too verbose, include implementation details, and leave out the rationale. We provide a replication package with the dataset, prompts, and scripts³.

2 Related Work

Early works in automated AKM, such as Bhat et al. [2] and Shahbazian et al. [17] show that issue discussions contain recoverable architectural knowledge, while tools like ADeX extract ADDs from project history artifacts [2]. However, these approaches depend on detailed textual documentation, which is often incomplete or unavailable in practice [21]. More recent work explores LLM-based support for architectural decision-making. Systems such as *ArchMind* and *Archify* assist architects using explicit descriptions of context and alternatives [7,15]. Other studies evaluate whether LLMs can generate ADRs from rich contextual inputs, existing ADRs, or high-level descriptions [19,1,6]. While these studies show that LLMs can generate structured architectural artifacts, they also report issues such as inconsistency and inappropriate abstraction levels [10]. To improve generation quality, later work focuses on prompting strategies and multi-stage pipelines. Maranhão et al. [14] use structured prompt sequences, while Rodriguez-Sanchez et al. [16] apply prompt and model ensembles for robustness. Other studies show that prompt design and contextual structure strongly affect ADR and rationale generation quality [6,22]. However, these approaches still rely on explicit high-level textual context rather than raw commit history.

In contrast, recovering ADDs directly from source code commits remains underexplored. This creates a practical gap because commits are often the only

³ Replication package – doi.org/10.5281/zenodo.20996340

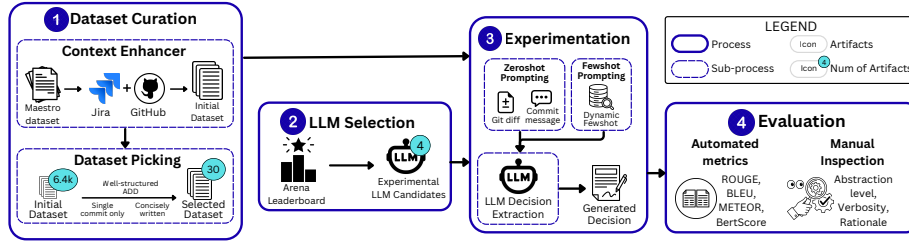


Fig. 1: Overview of Experimental Setup

consistently available record of architectural evolution. Our work addresses this gap by evaluating how effectively LLMs can recover ADDs from commits alone and how prompting and input structuring affect extraction quality.

3 Study Design

We conduct a preliminary study using a small dataset of developer-written ADDs from open-source systems. We use the term *alignment* to describe how closely a generated ADD reflects developer-documented ADD in terms of architectural intent, abstraction, and semantic meaning.

3.1 Research Questions

RQ₁: *To what extent do ADDs extracted by LLMs from source code commits using zeroshot prompting align with documented ADDs?* **RQ₂:** *How does fewshot prompting influence the alignment between extracted and documented ADDs?*

3.2 Experimental Setup

Dataset Curation We start from the Maestro dataset [13], which we choose as it already provides a mapping between Jira issues and the presence of ADDs, comprising 6,416 Jira tickets labelled for ADDs across 173 open-source projects. We link each ticket to its commit using the ticket ID found in commit messages, giving an initial dataset of (*commit*, *ADD*) pairs. From this, we manually

Table 1: LLMs used in the study.

Task	Model	Parameters	Provider	License
Experimental	Gemini 3 Pro	Unavailable	Google	Proprietary
	DeepSeek R1	685B	DeepSeek AI	Open-source
	Kimi K2 Thinking	1T (32B active)	Moonshot AI	Open-source
	Qwen3	32B	Alibaba Cloud	Open-source

picked 30 pairs that satisfy the following criteria: (1) the ADD is clearly written, (2) it maps to a single commit, and (3) the ADD description is between 60 and 100 words. These ADDs come from Apache Cassandra, Hadoop, and Tajo repositories, and cover technology choices, component-level changes, and quality-attribute changes. The dataset can be found in our replication package.

LLM Selection We pick four LLMs for this study, shown in Table 1. The models are selected from the Arena Leaderboard⁴, a widely accepted LLM benchmark [5]. We choose the models based on their ranking, size, and availability. Gemini 3 Pro is the top proprietary model, while DeepSeek R1 and Kimi K2 are the two largest open-source models. Qwen3 is the smallest model we can run locally.

Experimentation We compare two prompting strategies as depicted in Figure 1. *Zeroshot Prompting* is when the LLM is given the commit message and the *git diff*, and is asked to write the ADD. This approach evaluates the model’s ability to infer architectural intent directly from commit-level artifacts and distinguish architectural changes from implementation details. *Fewshot Prompting* provides the LLM with examples of previously documented ADDs from the initial dataset to guide generation toward expected architectural abstraction and structure. Relevant examples are dynamically retrieved using embedding-based similarity search over previous (*commit diff*, *ADD*) pairs. During inference, most semantically similar ADDs are included as in-context examples, enabling the model to better align outputs with architectural descriptions. The prompt template used is provided in our replication package.

Evaluation *Automated metrics.* We use established NLP similarity metrics to compare the extracted ADDs with the corresponding documented decisions. Specifically, we employ lexical metrics like ROUGE, BLEU, and METEOR, as well as the semantic metric BERTScore, which are widely used in text generation tasks [9]. Higher scores on these metrics indicate better alignment between extracted and documented design decisions. BERTScore typically yields higher baseline values since it measures semantic similarity rather than exact matches, so even small gains can reflect meaningful improvements in alignment.

Manual inspection. Automated metrics cannot tell apart real architectural reasoning from rewording of implementation details, so we complement them with a structured manual inspection. We defined a coding rubric *a priori* with three dimensions, each assessed against explicit criteria: *abstraction level* (whether the output describes the design decision or low-level implementation details), *verbosity* (whether its length is comparable to the documented ADD, or roughly 2× or more longer/shorter), and *rationale* (whether it explains *why* the change was made, not just *what* changed). One author applied this rubric to all 60 outputs of the best-performing model (Gemini 3 Pro), recording a judgement and a

⁴ Arena Leaderboard; accessed 2026-01-03 – arena.ai/leaderboard/text

short supporting note for each dimension. To limit subjectivity, the criteria were fixed before coding began and applied uniformly across all outputs.

4 Results

RQ₁: Zeroshot LLM alignment with documented ADDs

Automated metrics. As reported in Table 2, all four models reached a BERT-F1 above 0.81, which shows that LLMs can pick up architectural information from commits even without any task-specific help. Gemini was the best overall (BERT-F1 = 0.828, ROUGE-L = 0.134, BLEU = 0.013, METEOR = 0.175), followed by Qwen (BERT-F1 = 0.823). The word-overlap scores stayed low across all models, which means that the generated ADDs use different wording from the documented ones, even when they cover the same idea.

Manual inspection. The main problems observed were verbosity, wrong level of abstraction, and missing rationale. In about two thirds of the outputs, the generated ADD was 2 to 10 times longer than the documented one, as the model wrote a numbered list of small items instead of stating the decision. For example in **CASSANDRA-15505**, a one-line ADD about intercepting messages between nodes became a six-item list covering filter ordering, serialisation, and lifecycle changes. Most outputs also included at least one code-level detail (class names or configuration values). In **YARN-5889**, the documented ADD is to move user-limit computation out of the heartbeat path, but the generated ADD instead listed the classes involved (**AbstractUsersManager**, **UsersManager**). Many of the outputs also said *what* changed but not *why*. **CASSANDRA-3411** lists segment recycling, pre-allocation, and mmap I/O without saying these are done for commitlog performance.

Table 2: Performance comparison across the two approaches. Best-performing model for each metric within an approach is highlighted in bold.

Approach Model		ROUGE-L	BLEU	METEOR	BERT-F1	Support
Zeroshot	Kimi	0.092	0.003	0.155	0.814	30
	Deepseek	0.098	0.007	0.169	0.813	30
	Qwen	0.109	0.007	0.170	0.823	30
	Gemini	0.134	0.013	0.175	0.828	30
Fewshot	Kimi	0.104	0.009	0.180	0.827	30
	Deepseek	0.105	0.009	0.180	0.820	30
	Qwen	0.124	0.009	0.191	0.829	30
	Gemini	0.152	0.014	0.165	0.847	30

RQ₂: Impact of fewshot prompting on ADD alignment

Automated metrics. Fewshot prompting improved scores across all four models on most metrics, with the biggest gains on ROUGE-L and BERT-F1. Gemini remained the best overall (BERT-F1 = 0.847, ROUGE-L = 0.152, BLEU = 0.014). Qwen achieved the highest METEOR (0.191). The clear exception was METEOR for Gemini, which dropped from 0.175 to 0.165, suggesting that the retrieved examples sometimes push the wording further from the documented ADD even when the overall semantic match improves.

Manual inspection. About half of the very long zeroshot outputs were tightened to roughly the documented ADD’s length. CASSANDRA-15505 went from a six-item list to two sentences, HADOOP-14642 from a five-item list to a single paragraph, and YARN-5561 from a four-item REST-spec list to one sentence. The retrieved examples appear to teach the model the expected length and style of an ADD. However, implementation details still appeared in some outputs, and the missing-rationale problem was not addressed. The retrieved examples carry the documented ADDs but not the reasoning behind them, so they give the model no push to recover the *why* of a decision. Fewshot prompting therefore fixed the *structure* of the ADDs (length and style) but not their content, leaving the extraction of concise, well-grounded ADDs from commits an open problem.

5 Discussion

Key Learnings: *Commits show what changed, not why.* The model usually named changes well but left out the rationale for them. For example, YARN-5889 lists classes involved in the user-limit refactor but does not mention the heartbeat write-lock problem that motivated the change. This is a fundamental limitation of using commits as the sole input. The diff shows the change, but the reason for it usually lives in artifacts outside the commit, such as issue discussions, design documents, or code reviews. Commit messages sometimes hint at reason, but they are often short and code-focused, so the model has little to work with.

Difficulty in maintaining the correct level of abstraction. Most zeroshot outputs mentioned code-level items implementations class names and configuration values. Fewshot prompting made the outputs shorter, as seen in case of CASSANDRA-15505, but it did not consistently help to maintain the correct architectural abstraction. The retrieved examples teach the model the expected length and style of an ADD, but not how to abstract over a diff.

Implications for Researchers: These observations suggest that ADD extraction from commits is an architectural reasoning task rather than a pure summarisation task. Since commits primarily encode implementation changes, recovering the rationale likely requires combining commits with other artifacts where the reasoning is more explicit [18,2]. The results also reveal the limitations of automated similarity metrics, which cannot reliably distinguish architectural reasoning from implementation-level paraphrasing, highlighting the need for evaluation methods that better capture architectural meaning.

Implications for Practitioners: LLM-generated ADDs can be useful as a first draft for AKM, which can lower the documentation effort significantly. However, the output should be treated as a starting point and not as a finished ADD. The rationale for the decision usually needs to be added or checked, and implementation details should be removed.

6 Threats to Validity

Construct validity: The automated metrics (ROUGE-L, BLEU, METEOR, and BERTScore) may not capture architectural meaning, since the same ADD can be written with different words and at different levels of detail. We reduce this risk by combining all four metrics with a manual review. The manual inspection was done by a single author using free-text notes rather than a formal coding process with multiple reviewers, which can introduce subjectivity. We mitigate this by fixing the coding criteria before inspection and applying them uniformly to all outputs.

Internal validity: LLM outputs depend on how the prompt is worded. We reduce this risk by using the same prompt template for all runs. There is also a risk of data leakage, because the Apache projects we use are public and the corresponding commits and ADDs may appear in the LLMs’ pretraining data. We reduce this risk by combining the automated metrics with manual inspection focused on meaning rather than exact wording.

External validity: The dataset covers 30 ADDs from three Apache repositories (Cassandra, Hadoop and Tajo) and is biased towards Java-based systems. ADDs were picked for being clearly written and tied to a single commit, which introduces selection bias. Manual inspection covers only Gemini outputs so problems seen on weaker models may differ. The findings should be read as preliminary, and larger studies across more projects, languages, and models are needed.

7 Conclusion and Future Directions

We presented a preliminary exploratory study on extracting ADDs from source code commits using LLMs. LLMs can recover architectural information from commits to a moderate extent (BERT-F1 above 0.81 for all four models), and fewshot prompting improves the scores. However, the generated ADDs are often too implementation-focused and leave out the rationale behind the change. This suggests that extracting ADDs from commits is not purely a summarisation task, but a higher-level architectural reasoning problem. These limitations stem from the commit input itself and will likely persist as models evolve.

For future work, we plan to add other sources such as issue discussions, code reviews and repository-level context to help the model find the rationale better. Larger-scale evaluations and improved evaluation methodologies are needed to better assess architectural alignment and generation quality.

References

1. D. Amalfitano, M. De Luca, T. Santilli, P. Pelliccione, and A. R. Fasolino. Automated software architecture design recovery from source code using llms. In *ECSA*, pages 73–89, 2025.
2. M. Bhat, K. Shumaiev, A. Biesdorf, U. Hohenstein, and F. Matthes. Automatic extraction of design decisions from issue management systems: A machine learning based approach. In *ECSA*, pages 138–154, 2017.
3. R. Capilla, A. Jansen, A. Tang, P. Avgeriou, and M. A. Babar. 10 years of software architecture knowledge management: Practice and future. *JSS*, 116:191–205, 2016.
4. M. Chen et al. Evaluating large language models trained on code, 2021.
5. W.-L. Chiang, L. Zheng, et al. Chatbot arena: An open platform for evaluating llms by human preference, 2024.
6. R. Dhar, K. Vaidhyanathan, and V. Varma. Can llms generate architectural design decisions? - an exploratory empirical study. In *ICSA*, pages 79–89, 2024.
7. J. A. Díaz-Pace, A. Tommasel, and R. Capilla. Helping novice architects using an llm-based assistant. In *ECSA*, pages 324–332, 2024.
8. Z. Feng et al. CodeBERT: A pre-trained model for programming and natural languages. In *EMNLP Findings*, pages 1536–1547, 2020.
9. Y. Guo et al. Appls: Evaluating evaluation metrics for plain language summarization. In *EMNLP*, pages 9194–9211, 2024.
10. J. Jahić and A. Sami. State of practice: Llms in software engineering and software architecture. In *ICSA-C*, 2024.
11. A. Jansen and J. Bosch. Software architecture as a set of architectural design decisions. 2006.
12. R. Li et al. Starcoder: may the source be with you! 2023.
13. J. Maarleveld, A. Dekker, S. Druyts, and M. Soliman. Maestro: A deep learning based tool to find and explore architectural design decisions in issue tracking systems. In *ECSA*, pages 390–405, 2024.
14. J. J. Maranhão et al. A prompt pattern sequence approach to apply generative ai in assisting software architecture decision-making. In *EuroPLoP*, 2024.
15. B. C. Marinho, R. Bulcão-Neto, and V. V. G. Neto. Archify: A recommender system of architectural design decisions. 2021.
16. E. Rodriguez Sanchez, E. Vazquez-Santacruz, and H. Cervantes Maceda. A proposal for llm ensemble learning to achieve consensus in architectural design decisions. *SSRN*, 2025.
17. A. Shahbazian, Y. K. Lee, D. Le, Y. Brun, and N. Medvidovic. Recovering architectural design decisions. In *ICSA*, pages 95–109, 2018.
18. M. Soliman. Exploring architectural design decisions in mailing lists and their traceability to issue trackers. In *ECSA*, 2024.
19. M. Soliman, E. Ashraf, K. M. K. Abdelsalam, J. Keim, and A. P. S. Venkatesh. Llms for software architecture knowledge: A comparative analysis among seven llms. In *ECSA*, pages 99–115, 2026.
20. M. Soliman, K. Gericke, and P. Avgeriou. Where and what do software architects blog? : An exploratory study on architectural knowledge in blogs, and their relevance to design steps. In *ICSA*, 2023.
21. D. Tofan, M. Galster, and P. Avgeriou. Difficulty of architectural decisions: A survey with professional architects. In *ECSA*, pages 192–199, 2013.
22. X. Zhou, R. Li, P. Liang, B. Zhang, M. Shahin, Z. Li, and C. Yang. Using llms in generating design rationale for software architecture decisions. *TOSEM*, 2025.